

## INTRODUCTION

In your role as a software developer, you have been contracted by QUB's Photography Department to implement a Java application capable of managing a collection of photographic images for a range of genres. The Photography Department uses these images to produce a range of image Albums for display purposes. The image types and example images, with associated data, is given in the tables below.

### Image Types:

| Value               | Description                                                                                           |
|---------------------|-------------------------------------------------------------------------------------------------------|
| <b>ASTRONOMY</b>    | Photography or imaging of astronomical objects, celestial events, or areas of the night sky.          |
| <b>ARCHITECTURE</b> | Focuses on the capture of images that accurately represent the design and feel of buildings.          |
| <b>SPORT</b>        | Covers all types of sports and can be considered a branch of photojournalism.                         |
| <b>LANDSCAPE</b>    | The study of the textured surface of the Earth and features images of natural scenes.                 |
| <b>PORTRAIT</b>     | Images of a person or a group of people where the face and facial features are predominant.           |
| <b>NATURE</b>       | Focused on elements of the outdoors including sky, water, and land, or the flora and fauna.           |
| <b>AERIAL</b>       | Images taken from an aircraft or other airborne platforms.                                            |
| <b>FOOD</b>         | Captures everything related to food, from fresh ingredients and plated dishes to the cooking process. |
| <b>OTHER</b>        | Covers just about any other type of image and photography genre.                                      |

### Example Image Records & Associated Data:

| ID | Title             | Description                                    | Genre               | Date Taken | Thumbnail       |
|----|-------------------|------------------------------------------------|---------------------|------------|-----------------|
| 1  | Andromeda Galaxy  | Image of the Andromeda galaxy.                 | <b>ASTRONOMY</b>    | 01-01-2023 | Andromeda.png   |
| 2  | Lanyon QUB        | An image of the QUB Lanyon building.           | <b>ARCHITECTURE</b> | 01-02-2023 | LanyonQUB.png   |
| 3  | Kermit Plays Golf | An image of Kermit the frog playing golf.      | <b>SPORT</b>        | 01-03-2023 | KermitGolf.png  |
| 4  | Mourne Mountains  | A panoramic view of the Mourne mountains.      | <b>LANDSCAPE</b>    | 01-04-2023 | Mournes.png     |
| 5  | Homer Simpson     | Homer Simpson- A portrait of the man.          | <b>PORTRAIT</b>     | 01-03-2023 | Homer.png       |
| 6  | Red Kite          | A Red Kite bird of prey in flight.             | <b>NATURE</b>       | 01-04-2023 | RedKite.png     |
| 7  | Central Park      | An overhead view of Central Park New York USA. | <b>AERIAL</b>       | 01-05-2023 | CentralPark.png |
| 8  | Apples            | A bunch of apples.                             | <b>FOOD</b>         | 01-06-2023 | Apples.png      |
| 9  | Programming Meme  | A Chat GPT programming meme.                   | <b>OTHER</b>        | 01-07-2023 | ChatGPT.png     |

## GENERAL INSTRUCTIONS

1. Create a new Java project named 'Assessment2'.
2. Add two packages to 'Assessment2' named part01 and part02.
3. Implement your code as described below:
  - 'part01' should contain all code associated with the core requirements.
  - 'part02' should contain the code related to implementation of additional features only, including the use of class Console.

**NOTE: PLEASE READ THE FULL DOCUMENT BEFORE STARTING.**

**SUBMISSION DATE/TIME – FRIDAY 20<sup>TH</sup> MARCH 2026 BY 5 PM.**

## PART 1: CORE FUNCTIONALITY (70 MARKS AVAILABLE)

To demonstrate that the proposed Java application is feasible, the following must be implemented.

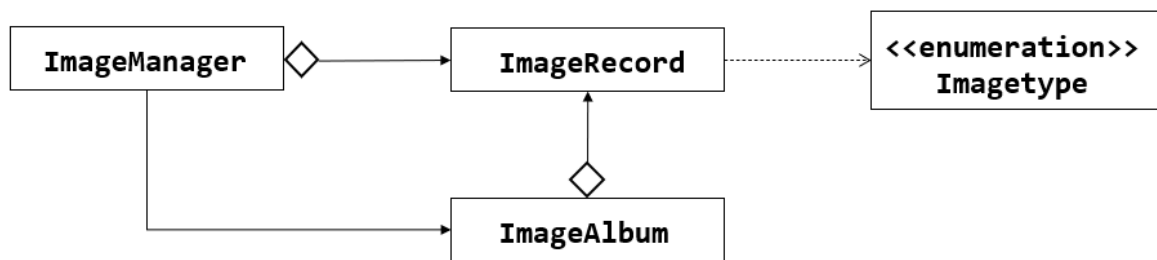


Figure 1 : Outline UML Diagram

1.1 Implementation of the components outlined in Figure 1 and detailed in Appendix 1.

1.2 A standard text console (menu-based) application (called **QUBImages**) to provide system behaviour. The following general menu options should be provided:

- **Add Image**- adds a new **ImageRecord** to an **ImageManager** instance; must ask user for required data.
- **Search** – should provide sub-options for searching by **unique id**, **title**, **description**, **type**, **date range** – search results should be displayed in an appropriate format.
- **Display All** – should display details of all images within the system, appropriately formatted.
- **Exit**: the user should be able to exit the system.

### Notes:

- A. **ImageRecord** objects should be **uniquely identifiable** within the system (using an integer id).
- B. The system application should make use of the standard console **Menu** class described in the lectures (and made available on Canvas).
- C. By default, a range of **ImageRecord** objects should be available within the system on startup.
- D. You are restricted to **Java core language features** described at the start of the module and used in the code examples.
- E. You may use **Scanner** and **ArrayList** (restricted to **add**, **get**, **size** methods only). You may also use **LocalDate**.
- F. You are not permitted use of any other library.

## PART 2: ADDITIONAL FUNCTIONALITY (30 MARKS)

This part of the assessment uses a new class (**Console**) to replace the standard system output console. Information on how to use class **Console** is available in the material for **Practical 5** and **Appendix 2**.

Using the **Console** class, implement a new application called **QUBMediaImages** which replicates the behaviour of **QUBImages** from **Part 1**. Specifically, the application should:

- Make use of **Console** instance(s) for all input and display functionality.
- Use images, fonts and colour as appropriate to enhance the user experience – add a standard folder called 'Images' to the project – to contain all images files.
- Apply the same coding restrictions outlined for Part 1 above, aside from using **CSC1029Console.jar** containing the definition of class **Console**.

### Notes:

Marks will be allocated for:

- functionality/behaviour
- creativity in the use of the **Console** class.
- code reuse.

## CHAT GPT

You are required to provide a brief report indicating where and how you have used Chat GPT in your work for this assessment. Include the following information for part 1 and 2:

1. Version of Chat GPT used,
2. Details of use (e.g. for code generation, debugging)

If have not used Chat GPT in any way – please state this in your (very brief) report.

## SUBMISSION INSTRUCTIONS

1. Create a new folder to hold all your assignment files for submission. The folder should be suitably named and include your student number.
2. Copy and paste the 'part01' and 'part02' folders from your java project into the folder created in step 1. ENSURE that the 'part01' and 'part02' folders contain all of the corresponding .java files.
3. Copy and paste your 'Images' folder to you submission folder
4. Include the brief report (on the use of ChatGPT) and the Images folder (used in Part 2)
5. Compress the submission folder to a zip file and upload it to Canvas by **5.00pm on Friday 20th March 2026.**

**NOTE:** Please check that you have included the original development Java (.java) files. If you submit .class files these will NOT be marked. These submissions will be date-stamped and in accordance with University regulations, late submissions will be penalised. **Failure to follow the submission instructions may result in a reduced or zero mark for part or all of the assignment!**

## APPENDIX 1: COMPONENT IMPLEMENTATION DETAILS

For the system object classes, the internal attributes (class & instance data) are for you to decide upon and implement. However, each class should provide the following (public) behaviour, *precisely as indicated*:

### Object Class ImageRecord – Describes the attributes & behaviour of ImageRecord objects

- Accessors (getters) for all instance data values
- Mutators (setters) for all appropriate instance data values
- A **toString** method which return a String (no line breaks) with details of all instance values, appropriately formatted.

### Object Class ImageManager – An instance of this class will be responsible for managing a collection of (unique) ImageRecord instances.

- **public void addImage(ImageRecord image)** – adds a reference to an **ImageRecord** object to the collection.
- **public ImageRecord searchId(int id)** – returns a reference to an **ImageRecord** object (identified by **id**) or **null**.
- **public ImageAlbum searchTitle(String str)** – returns a reference to an **ImageAlbum** instance for images with **title** containing the **String str**.
- **public ImageAlbum searchDescription(String str)** – returns a reference to an **ImageAlbum** instance for images with **description** containing the **String str**.
- **public ImageAlbum searchGenre(ImageType type)** – returns a reference to an **ImageAlbum** instance for images which match **type**.
- **public ImageAlbum searchDates(LocalDate start, LocalDate end)** – returns an **ImageAlbum** for images taken between **start** and **end** dates.
- **public ImageAlbum getAllImages()** – returns a reference to an **ImageAlbum** instance for all images managed by an **ImageManager** object.

### Object Class ImageAlbum – An instance of this class will be generated by an ImageManager as a result of a search and will be responsible for a collection of ImageRecord instances. All ImageRecords managed by an ImageAlbum object should be held in date order (earliest first) and should be accessed using the following methods:

- **public ImageRecord getFirst()** – returns a reference to an **ImageRecord** instance – the first or ‘earliest’ image in an **ImageAlbum** or **null** if there are no images in the album.
- **public ImageRecord getNext()** – returns a reference to an **ImageRecord** instance – the image ‘next’ to the last image accessed – or **null** if no images in the album or no ‘next’ image exists or no images previously accessed.
- **public ImageRecord getPrevious()** – returns a reference to an **ImageRecord** instance – the image ‘previous’ to the last image accessed - or **null** if no images in the album or no ‘previous’ image exists or no images previously accessed.
-

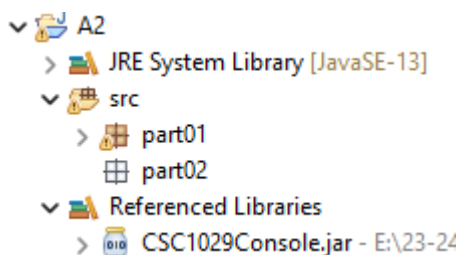
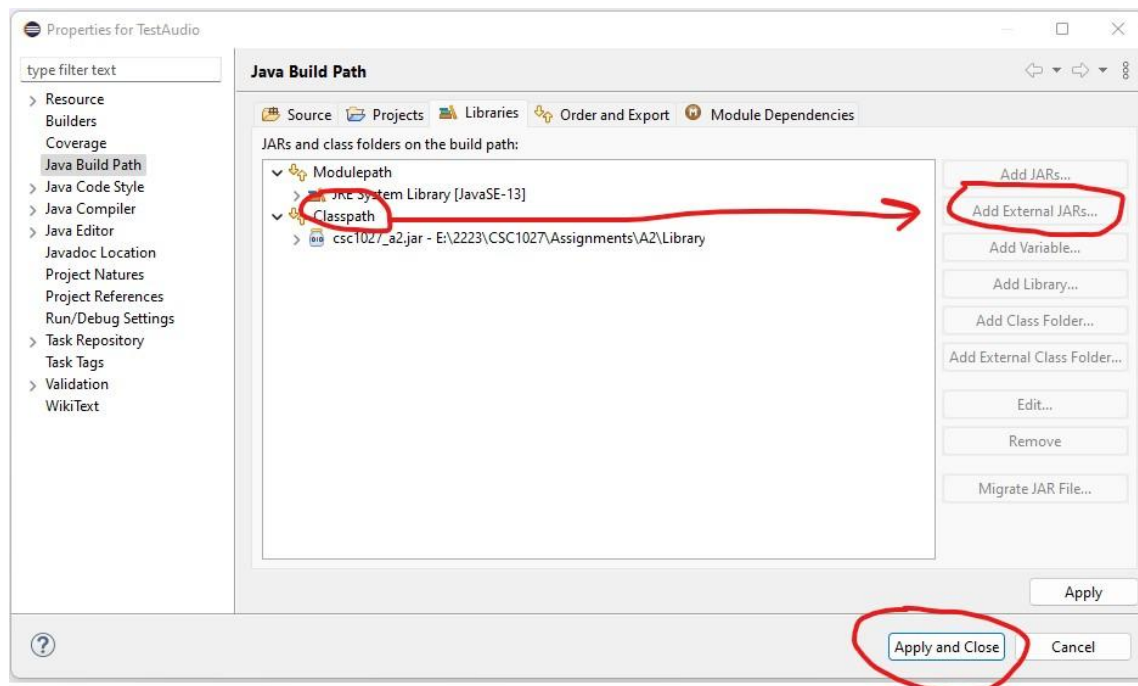
## APPENDIX 2: HOW TO INSTALL THE EXTERNAL LIBRARY FILE (JAR)

**Step 1** – Download the following files from Canvas (under Assessment 2)

- **CSC1029Console.jar** – this file contains the definitions for classes **Console**

**Step 2** – Add the downloaded jar file to your project:

- From the package explorer in Eclipse, “**right-click**” your project directory and select “**Build Path->Configure Build Path ..**”
- Select the “**Libraries**” tab and highlight “**Classpath**”
- Click the “**Add External Jar**” button (on the right-hand side) and navigate to the downloaded Jar file to select/open.
- Click on the “**Apply and Close**” button



<-- Your Package Explorer should look something like this

<-- The Jar file should be visible within “Referenced Libraries”

### Console Methods:

- constructor** – takes a single boolean parameter: true (console supports user input), false (input not supported)
- clear()** – will clear the console of text and images
- println()** – overloaded to allow for output of text, images (ImageIcon) and other objects.
- getFont()** – returns the current Font in use by the console.
- setFont()** – parameter defines the Font to be used by the console.
- setBgColour()** – parameter defines the colour used for the console background.
- setColour()** – parameter defines the colour used within the console for text.
- readLn()** – returns a String in response to user typing text within the console (note that the console must be initialised to permit this).

| Console                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div><div>+Console(boolean)</div><div>+clear():void</div><div>+print(Object):void</div><div>+println(Object):void</div><div>+println():void</div><div>+println(String):void</div><div>+print(String):void</div><div>+getFont():Font</div><div>+setFont(Font):void</div><div>+setBgColour(Color):void</div><div>+setColour(Color):void</div><div>+readLn():String</div></div></div> |